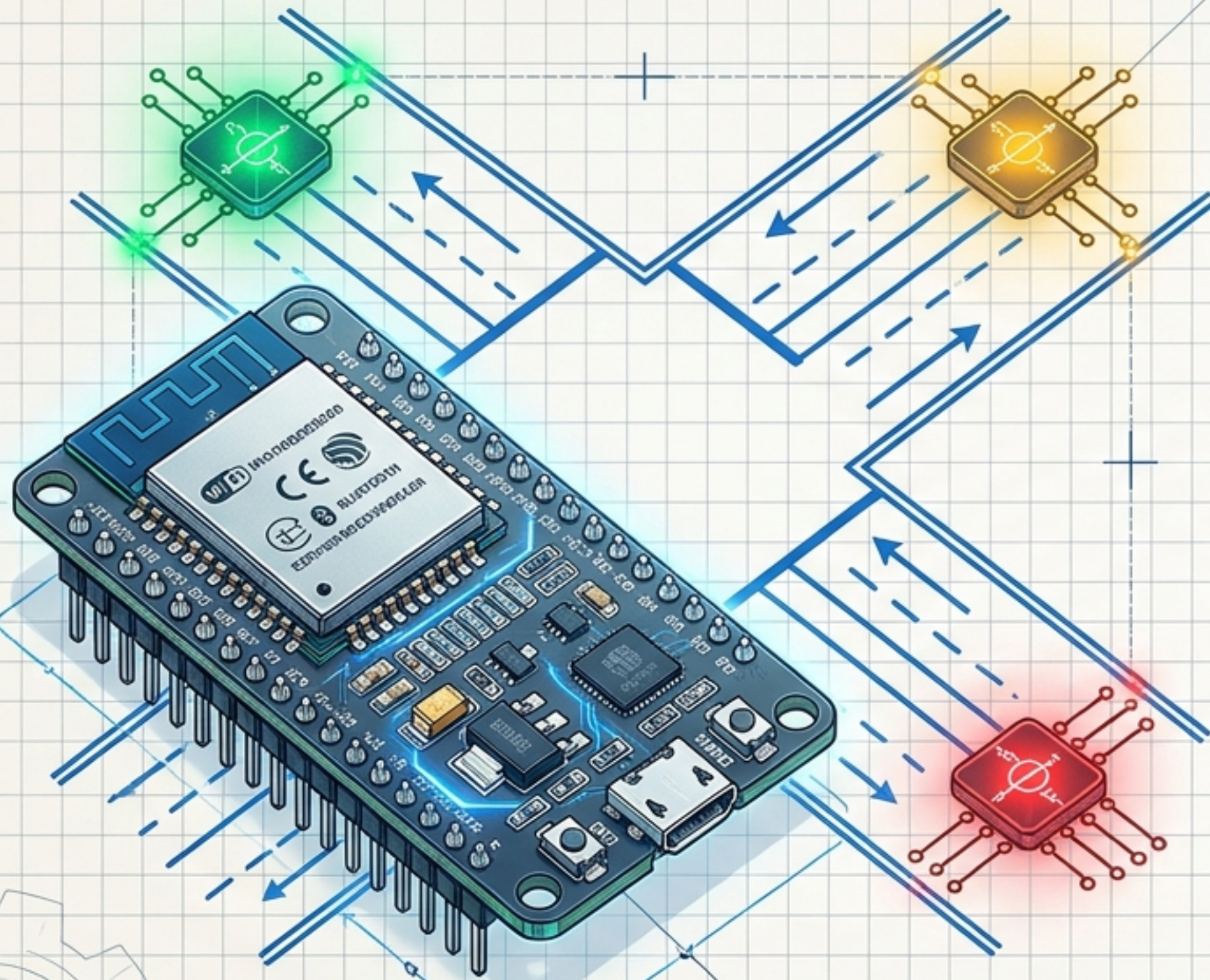




ENGINEERING
MANUAL



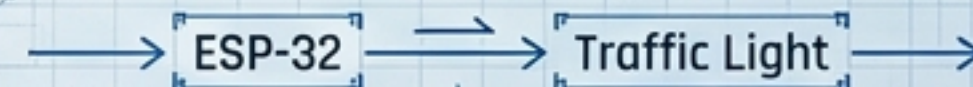
DYNAMIC
BLUEPRINT
SYSTEM



ESP-32

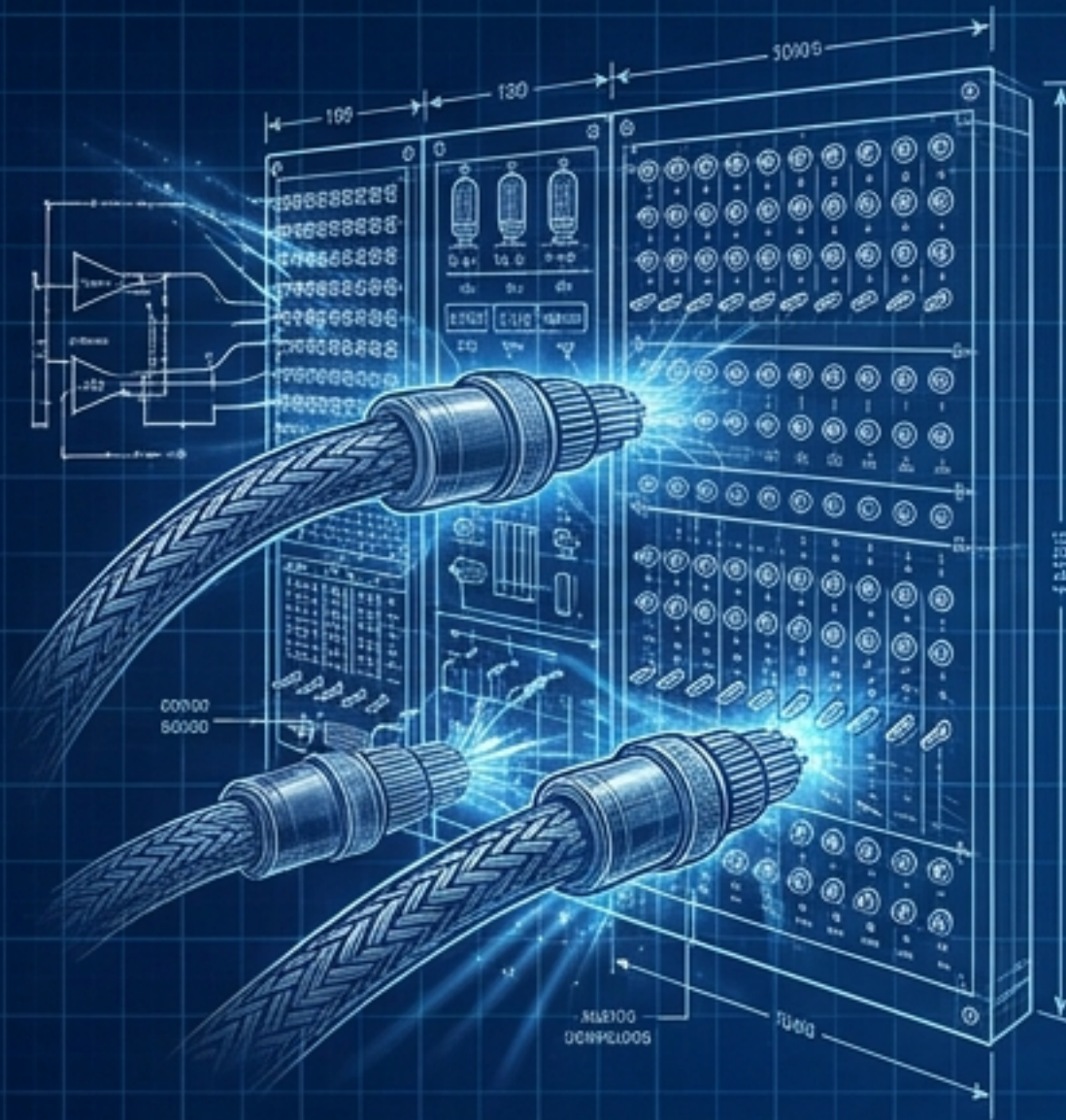
Aula 02: Do Código ao Cruzamento

Desenvolvendo um controlador de semáforo determinístico com ESP-32 e desvendando a camada de hardware.



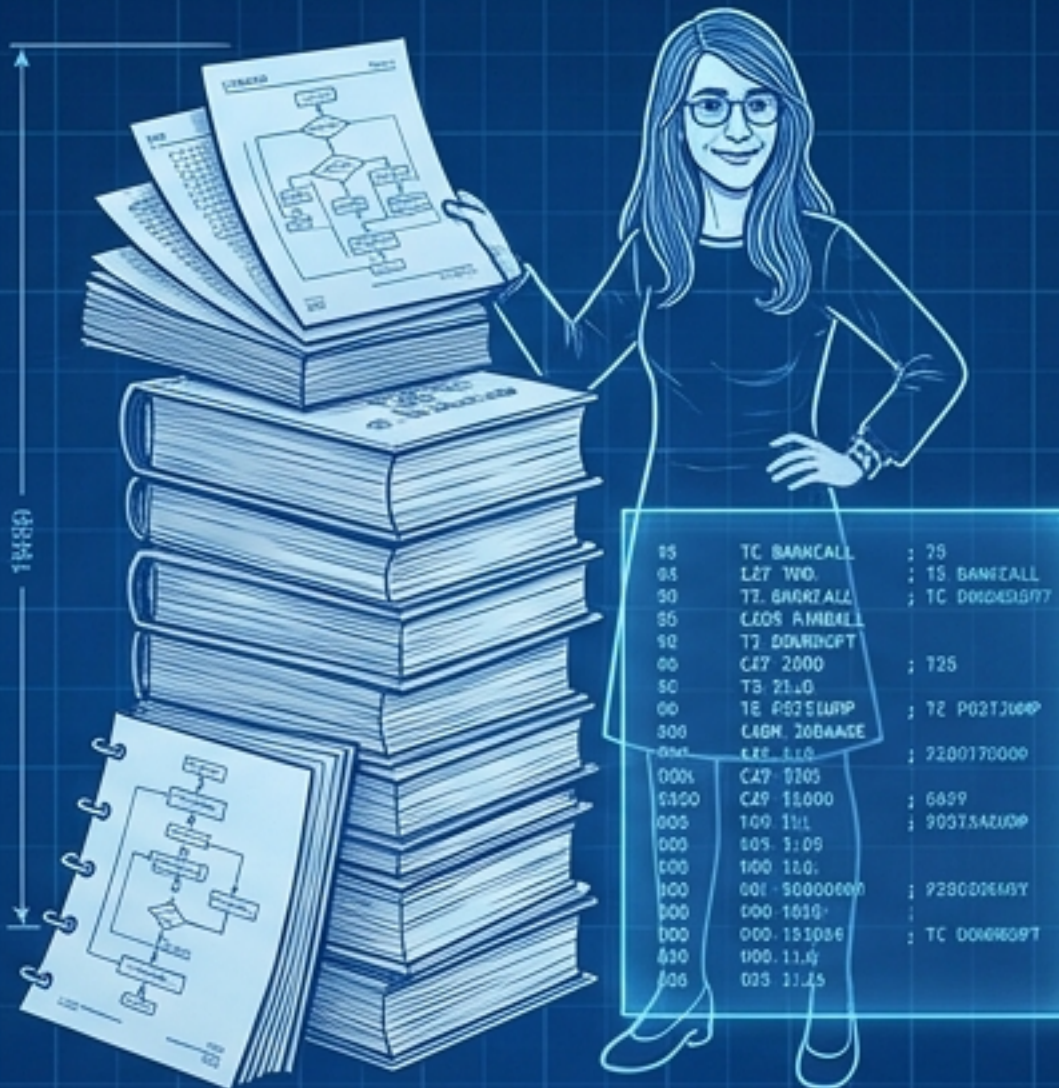
ESP-32

A abstração é o nosso superpoder, mas um engenheiro conhece o hardware.



1946 - ENIAC

Programação Física (Hardware direto)



1969 - Apollo 11

Instruções de Máquina (Assembly)



Hoje - ESP32

Abstração de Alto Nível (C++)

**Do cabo ao C++, a lógica permanece.
Hoje, conectamos o software ao mundo físico.**

O Desafio do Dia: Orquestrando o Caos

Metas da Engenharia:

1. Especificar e atender requisitos.
2. Descer ao nível do compilador.
3. Criar uma Máquina de Estados determinística.
4. Documentar a arquitetura.

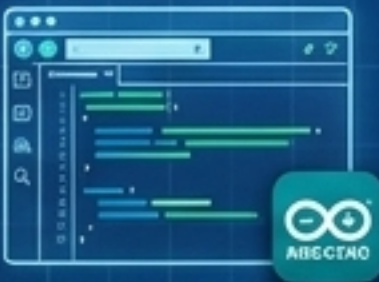
O Arsenal do Laboratório de Processadores



Versionamento: GitHub
(Controle de versões e portfólio)



Análise: Inteligência Artificial
(Engenharia reversa do Assembly)



Software: Arduino IDE 2.x
(Ambiente C/C++)

Hardware:
Placa ESP-32
(A ponte física)



Nota: Os componentes estarão disponíveis na Sala C1-13. Requisitar no início da aula.

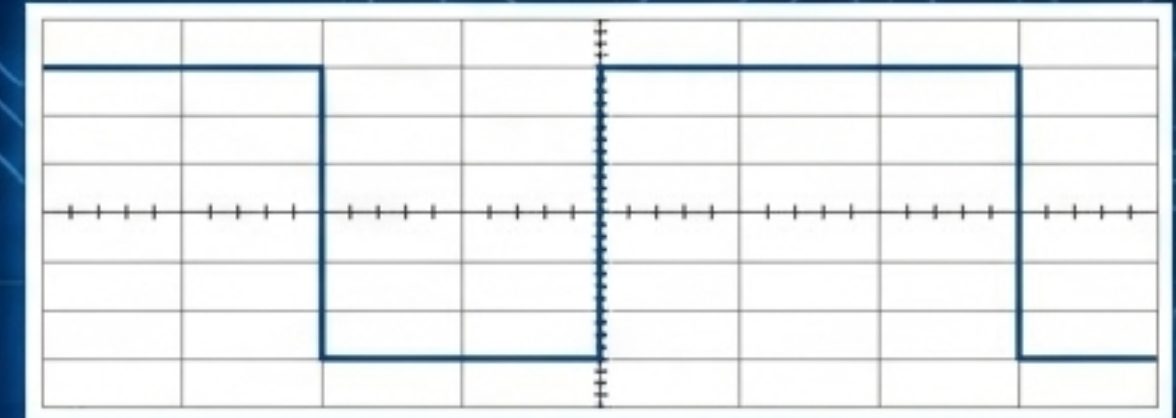
Missão 1: Setup da Infraestrutura



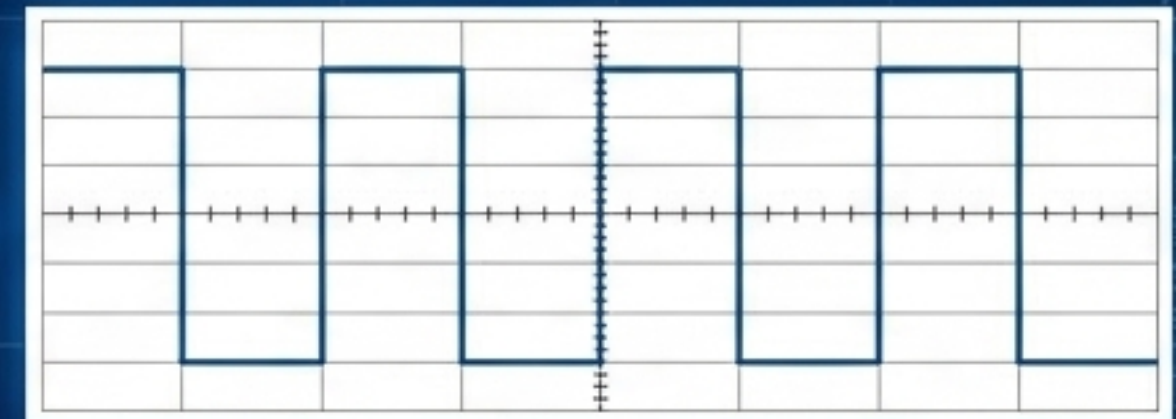
O exemplo de tutorial está no moodle.

Missão 2: O Hello World Físico

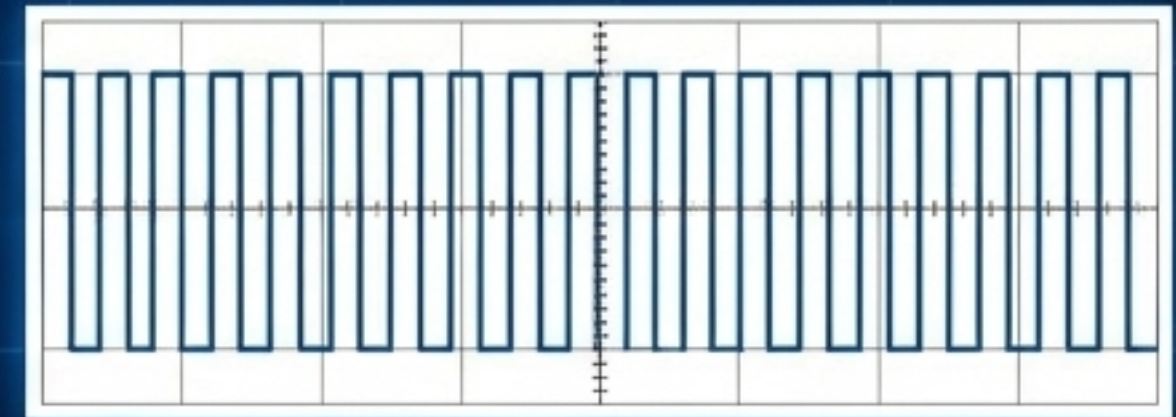
```
void setup() {  
  pinMode(LED_BUILTIN, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(LED_BUILTIN, HIGH);  
  delay(T);  
  digitalWrite(LED_BUILTIN, LOW);  
  delay(T);  
}
```



Onda 1: delay(1000) - Pisca Lento



Onda 2: delay(200) - Pisca Rápido



Onda 3: delay(50) - Frequência de Atenção

Missão 3: Dissecando a Caixa Preta do Compilador



Humano

Blink.ino (Código C++)



Processo de Compilação

Sketch -> Export Compiled Binary



Máquina

Blink.elf (Machine Code)

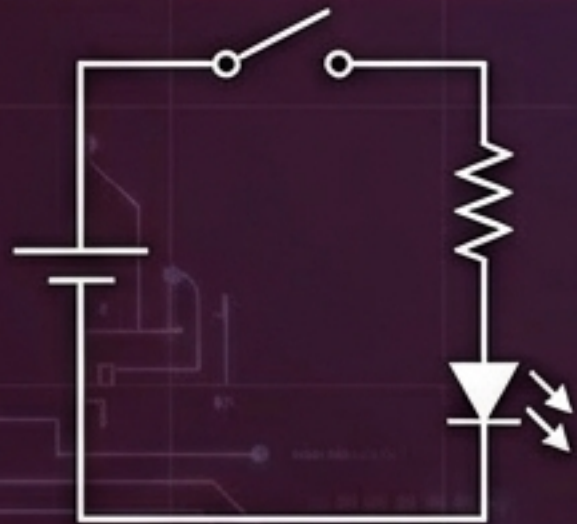
O Raio-X da GenAI: C++ vs Assembly

C++ Abstrato

```
pinMode(LED_BUILTIN, OUTPUT);  
digitalWrite(LED_BUILTIN, HIGH);
```

Instruções de Hardware

```
li a5, 1  
sw a5, 0(a4)  
jal ra, digitalWrite
```



A Pergunta Fundamental:

Ao executar
`digitalWrite(LED_BUILTIN, HIGH)`,
o LED acende ou apaga?
Lógica Active-High vs Active-Low

Missão 4: Estado de Atenção (Semáforo de Garagem)



Antes de orquestrar a sequência de um cruzamento complexo, dominamos o controle intermitente de um único canal de hardware.

Missão 5: A Máquina de Estados do Cruzamento



A arquitetura de software para sistemas físicos exige tempo estrito e determinismo contínuo.

Engenharia exige rigor: **O Relatório e as Evidências**

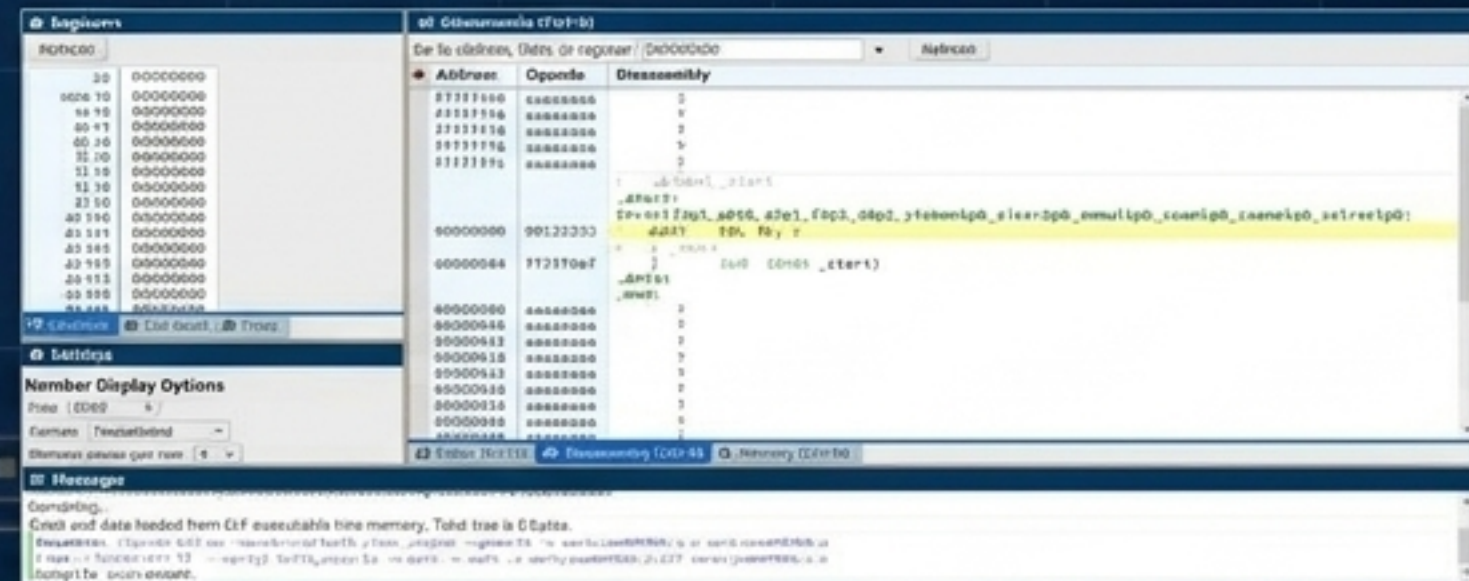
Regras:

Máximo 10 páginas
Fonte Arial 12, formato PDF

Seção 3 - Requisitos:

Tabela detalhando RFs e RNFs.

Evidências e Testes:



The screenshot shows the CPUlator simulator interface. On the left, a 'Registers' window lists various registers (R0, R1, etc.) with their current values. On the right, a 'Disassembly' window shows the assembly code being executed, with instructions like 'LD R0, #0' and 'LD R1, #0' highlighted. The interface includes tabs for 'Registers', 'Disassembly', and 'Memory'.

Address	OpCode	Disassembly
00000000	00000000	LD R0, #0
00000001	00000000	LD R1, #0
00000002	00000000	LD R2, #0
00000003	00000000	LD R3, #0
00000004	00000000	LD R4, #0
00000005	00000000	LD R5, #0
00000006	00000000	LD R6, #0
00000007	00000000	LD R7, #0
00000008	00000000	LD R8, #0
00000009	00000000	LD R9, #0
0000000A	00000000	LD R10, #0
0000000B	00000000	LD R11, #0
0000000C	00000000	LD R12, #0
0000000D	00000000	LD R13, #0
0000000E	00000000	LD R14, #0
0000000F	00000000	LD R15, #0

Exemplo: Tabela de Testes do simulador CPUlator validando o Assembly e os registradores.

Mapeando o Método e a Arquitetura

Seção 4: Arquitetura

Lógica de Controle
(C++)

GPIO
ESP-32

LEDs
Físicos

Seção 5: Método

Setup

Implementação

Teste

Falha?

Correção no
Código/Hardware

Seção 6: Lições Aprendidas

Não é um diário; é uma análise de engenharia sobre as falhas e como o método foi refinado durante a aula.

O Repositório Vivo: **Repositório Obrigatório (Versionamento Opcional)**

- Node 1: "init: setup do projeto ESP32"
- Node 2: "feat: adiciona teste de blink no built-in led"
- Node 3: "feat: implementa logica de atencao amarela"
- Node 4: "feat: finaliza maquina de estados do cruzamento"

"Seu código é seu portfólio. O relatório exige o link público para o repositório GitHub contendo todo o código desenvolvido."

Checklist



Inferência de IA
(ARM/RISC-V)



Aceleradores
Criptográficos



IoT / Freenove



Exploração
(GitHub/Pinterest)



Ambiente ESP32 configurado.



Blink testado e binário .elf
analisado por GenAI.



Semáforo de Atenção (Amarelo)
rodando.



Máquina de Estados
(Verde/Amarelo/Vermelho) finalizada.



Repositório público no GitHub
atualizado.



Relatório PDF finalizado com
diagramas e lições aprendidas.



Uma ideia para o projeto da disciplina.